



YEDİTEPE UNIVERSITY

Faculty of Engineering

Department of Computer Engineering

Term Project Report

CSE480/591: Optimization with Metaheuristics

Student Name: Atahan Düryaz, Derin Taşkın

Student ID: 20220702085

Date: November 23, 2025

Project Title:

Solving the Traveling Salesman Problem using Metaheuristic Approaches

Instructor: Asst. Prof. Dr. Gizem Süngü Terci

Fall 2025 Semester

1. Selected Problem and Motivation

The selected optimization problem for this project is the **Traveling Salesman Problem (TSP)**. The TSP is one of the most studied NP-hard combinatorial optimization problems in operations research and computer science. The core objective is to determine the shortest possible route that visits a specific set of cities exactly once and returns to the origin city.

The motivation for selecting the TSP lies in its theoretical significance and its wide range of real-world applications.

- **Real-World Relevance:** Variations of the TSP are fundamental to logistics (vehicle routing), manufacturing (drilling of printed circuit boards), and telecommunications (Talbi, 2009).
- **Computational Complexity:** As the number of cities (n) increases, the search space grows factorially ($n!$), rendering exact methods like brute-force or simple branch-and-bound computationally expensive or infeasible for large-scale instances.
- **Benchmark Standard:** The TSP serves as a standard benchmark for evaluating the performance of new metaheuristic algorithms.

Consequently, this project aims to explore **metaheuristic** approaches (e.g., Evolutionary Algorithms or Swarm Intelligence) to find near-optimal solutions efficiently, rather than relying on exact methods or simple greedy heuristics.

2. Formal Problem Definition

Formally, the TSP is defined on a complete graph $G = (V, E)$, where $V = \{1, 2, \dots, n\}$ represents the set of cities (vertices) and $E = \{(i, j) : i, j \in V, i \neq j\}$ represents the set of edges. Each edge (i, j) is associated with a cost (distance) d_{ij} .

To satisfy the requirement for a rigorous optimization model, we define the problem using an **Integer Linear Programming (ILP)** formulation. This mathematical model provides the foundation upon which metaheuristics operate by defining the search space and feasibility constraints.

2.1 Decision Variables

We utilize binary decision variables x_{ij} to represent the selection of edges in the tour:

$$x_{ij} = \begin{cases} 1 & \text{if the tour moves directly from city } i \text{ to city } j \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

for all $i, j \in V$, where $i \neq j$.

2.2 Objective Function

The objective is to minimize the total cost of the tour. The mathematical formulation is:

$$\text{Minimize } Z = \sum_{i=1}^n \sum_{j=1, j \neq i}^n d_{ij} x_{ij} \quad (2)$$

Here, d_{ij} represents the weight (distance or cost) of the edge between city i and city j .

2.3 Constraints

To ensure the solution forms a valid Hamiltonian cycle (a single tour visiting all cities), the following constraints must be satisfied. We present the constraints based on the integer programming formulations discussed in [Benhida and Mir \(2018\)](#).

- (i) **Assignment Constraints (Degree Constraints):** Each city must be entered exactly once and left exactly once.

$$\sum_{j=1, j \neq i}^n x_{ij} = 1 \quad \forall i \in V \quad (\text{Out-degree}) \quad (3)$$

$$\sum_{i=1, i \neq j}^n x_{ij} = 1 \quad \forall j \in V \quad (\text{In-degree}) \quad (4)$$

- (ii) **Sub-tour Elimination Constraints (SECs):** Constraints (i) alone allow for multiple disconnected loops (sub-tours). To prevent this, specific constraints are required. Literature provides different formulations for SECs, notably the Danzig-Fulkerson-Johnson (DFJ) and Miller-Tucker-Zemlin (MTZ) formulations ([Benhida and Mir, 2018](#)).

Option A: DFJ Formulation (General Set-Based): The classical constraint requires that for any proper subset of cities S , the tour cannot be closed within S :

$$\sum_{i \in S} \sum_{j \in S, i \neq j} x_{ij} \leq |S| - 1 \quad \forall S \subset V, 2 \leq |S| \leq n - 1 \quad (5)$$

This formulation is mathematically strong but generates an exponential number of constraints.

Option B: MTZ Formulation (Polynomial): Alternatively, [Benhida and Mir \(2018\)](#) highlight the Miller-Tucker-Zemlin (MTZ) formulation, which uses auxiliary variables u_i (representing the order of visit) to eliminate sub-tours using a polynomial number of constraints ($O(n^2)$):

$$u_i - u_j + nx_{ij} \leq n - 1 \quad \forall i, j \in \{2, \dots, n\}, i \neq j \quad (6)$$

$$1 \leq u_i \leq n - 1 \quad \forall i \in \{2, \dots, n\} \quad (7)$$

This formulation is suitable for defining the problem in a compact optimization format.

(iii) **Integrality Constraints:**

$$x_{ij} \in \{0, 1\} \quad \forall i, j \quad (8)$$

3. Dataset or Benchmark Instances

To evaluate the proposed metaheuristic approach, we will use the standard **TSPLIB** benchmark library (Reinelt, 1991), which is the universally accepted dataset for comparing TSP algorithms.

- **Data Source:** TSPLIB (University of Heidelberg).
- **Instance Selection:** We plan to use a diverse set of instances to test scalability:
 - Small/Medium: `eil51`, `berlin52`, `st70`.
 - Large: `ch130`, `pr152`.
- **Format and Calculation:** The instances typically provide 2D coordinates. Consistent with TSPLIB conventions, distances are calculated as rounded Euclidean distances:

$$d_{ij} = \text{nint} \left(\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \right)$$

4. Example Scenario / Problem Instance

To concretize the formal optimization model defined in Section 2, we present a small-scale, manually verifiable instance. This instance serves as a preliminary test bed to verify the implementation of the metaheuristic's components.

4.1 Instance Data

We define a problem with $n = 5$ cities. Table 1 provides the Cartesian coordinates, which define the spatial layout of the cities.

While the coordinates visualize the problem, the **metaheuristic algorithm operates on the edge weights**. Therefore, we compute the full Symmetric Distance Matrix (Table 2) using Euclidean distances. This matrix represents the d_{ij} costs that the algorithm will query to calculate the fitness of any candidate solution.

Table 1: Coordinates of the 5-city instance.

City ID	X-Coordinate	Y-Coordinate
1	0	0
2	4	0
3	4	3
4	0	3
5	2	1.5

Table 2: Symmetric Distance Matrix (The Input for the Optimization Algorithm).

	1	2	3	4	5
1	0.00	4.00	5.00	3.00	2.50
2	4.00	0.00	3.00	5.00	2.50
3	5.00	3.00	0.00	4.00	2.50
4	3.00	5.00	4.00	0.00	2.50
5	2.50	2.50	2.50	2.50	0.00

4.2 Visual Representation

The spatial configuration of the instance, including city coordinates and a potential optimal tour, is illustrated in Figure 1. The red arrows visualize the optimal path, with edge weights (distances) annotated on each segment.

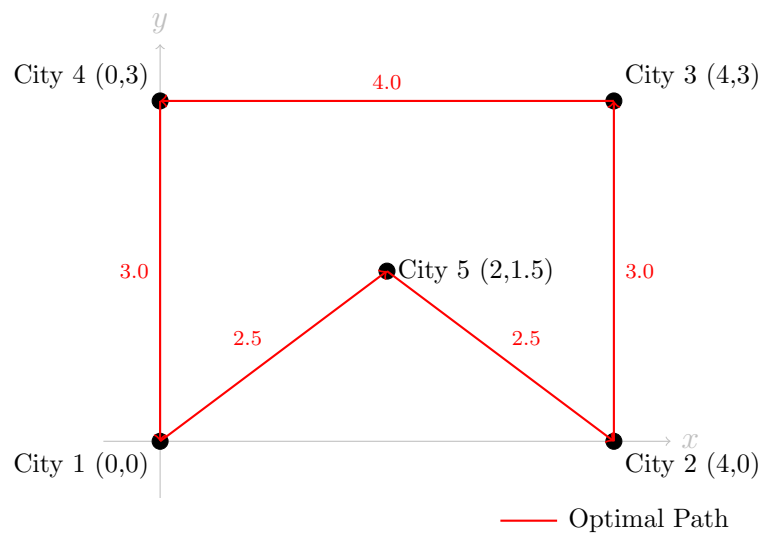


Figure 1: Visual representation of the 5-city problem instance. The red path indicates an optimal tour (Total Cost = 15.0).

4.3 Justification of the Instance

This specific instance is chosen to be representative of the larger optimization challenges for three reasons:

1. **Clear Structure:** It represents a standard fully connected graph with Euclidean distances, matching the TSPLIB format.
2. **Trap for Greedy Heuristics:** The central node (City 5) is equidistant ($d = 2.5$) to all four corner cities. A simple Nearest Neighbor heuristic starting at City 1 might choose City 5 first, but then be forced to jump to a corner (e.g., City 2) and traverse the perimeter, which might not always lead to the global optimum compared to strategies that plan the center visit more carefully.
3. **Verifiability:** The optimal tour (e.g., $1 \rightarrow 5 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1$) has a calculated cost of **15.0**. This allows us to strictly validate the correctness of the objective function calculation in the implementation phase before running the algorithm on larger, unknown datasets.

5. Proposed Approach

This section with its subsections belongs to the third progress of the term project... Do not consider now.

5.1 Overview Design and/or General Algorithm

5.2 Component 1 of Your Algorithm

5.3 Component 2 of Your Algorithm

6. Experimental Study

This section with its subsections belongs to the fourth progress of the term project... Do not consider now.

6.1 Experimental Setup

6.1.1 Test Environment

6.1.2 Parameter Setting

6.2 Experimental Results

7. Discussion and Conclusion

This section belongs to the final report of the term project... Do not consider now.

8. References

References

- Benhida, S. and Mir, A. (2018). Generating subtour elimination constraints for the traveling salesman problem. *IOSR Journal of Engineering (IOSRJEN)*, 8(7):17–21.
- Reinelt, G. (1991). TspLib—a traveling salesman problem library. *ORSA Journal on Computing*, 3(4):376–384.
- Talbi, E.-G. (2009). *Metaheuristics: From Design to Implementation*. Wiley, Hoboken, NJ.